



An energy-efficient dehazing neural network accelerator based on E²AOD-Net

Zhihao Zhang¹ · Gaoming Du¹ · Zhenmin Li¹ · Qinran Kang¹ · Wenyao Zhao¹ · Xiaolei Wang¹

Received: 5 May 2024 / Accepted: 21 October 2024 / Published online: 15 November 2024
© The Author(s), under exclusive licence to Springer-Verlag GmbH Germany, part of Springer Nature 2024

Abstract

Turbid media such as fog and haze seriously affects the quality of imaging for systems such as urban surveillance and satellite remote sensing. Image dehazing has become a research hotspot in the field of computer vision. Neural-network-based image dehazing has the potential of high performance, but requires high computational power and storage space, making it costly to deploy in a system with limited hardware resources, especially for edge computing systems. In this paper, we propose an energy-efficient dehazing neural network named E²AOD-Net, which is pruned (generalization performance rises), quantized, pipelined, and parallelized over AOD-Net. We implement E²AOD-Net on FPGA platform, achieving a lightweight dehazing hardware accelerator that realizes real-time dehazing. The experimental results show that the frame rate of E²AOD-Net hardware accelerator inference reaches 38.3 FPS, while consuming the power of 2.491 watts. The VMAF index is improved by 89.14%. The energy efficiency is 42.61 GOPS/w.

Keywords Image dehazing · Neural networks · FPGA · Real-time processing · Energy efficiency

1 Introduction

In turbid media such as fog and haze, the light received by the camera is weakened or altered, combining the interference from atmospheric light, the quality of imaging can be deteriorated seriously, which in turn lower the performance of subsequent image processing tasks. To tackle this issue, image dehazing techniques are vital as they can significantly improve the robustness of imaging quality and guarantee the effectiveness of the subsequent visual processing tasks.

There are three major research directions in the arena of image dehazing, which are based on image enhancement, physical model, and machine learning, respectively. The dehazing methods based on image enhancement use traditional image processing methods such as histogram equalization, wavelet transform, and homomorphic filtering, but have poor adaptability to the shape and concentration of the turbid media. The dehazing methods based on physical

model analyze physical imaging mechanism of foggy days, estimate the atmospheric light and refractive index through mathematical modeling, and achieve dehazing by solving the equations inversely [7]. The dehazing methods based on machine learning exploit machine learning techniques to fit the physical parameters in the equations, with the highlight of automatic feature extraction and end-to-end training. In general, machine learning methods can achieve better dehazing results compared to image enhancement and physical model methods. Admittedly, neural networks have higher computational complexity and larger storage requirements. Edge computing systems are typically equipped with limited computing capability, storage space and power consumption. They also have higher real-time requirements for image processing. Therefore, the end-side deployment of dehazing algorithms using neural networks faces severe challenges in terms of both algorithm design and hardware implementation.

To achieve efficient real-time dehazing and low resource consumption in multiple scenarios, we propose a real-time image dehazing system based on PYNQ which is implemented by the method of software-hardware co-design. We implement the heterogeneous computing system on

✉ Zhenmin Li
zhenmin.li@hfut.edu.cn
Gaoming Du
dugaoming@hfut.edu.cn

¹ School of Microelectronics, Hefei University of Technology, Hefei, China

PYNQ-Z2.¹ We also propose and utilize various hardware design methods based on neural network hardware implementation and design a reusable neural network IP (Intellectual Property). After comprehensively considering the dehazing effect, computational complexity, and hardware resource overhead, we choose the AOD-Net structure as the basis for improvement and propose E²AOD-Net (Energy-Efficient AOD-Net). We package the E²AOD-Net IP core deploy it on the PL (Programmable Logic), and use it as a co-processor of PS (processing system) to accelerate the convolutional computation. Finally, we test and compare the system's power consumption, speed, and other metrics, build a video test bench, perform VQA (video quality assessment) on the output real-time dehazing video, and compare the difference in metric values between the software simulation and the hardware implementation of the results.

The main contributions of this paper are as follows:

- We propose a new model structure named E²AOD-Net (Energy-Efficient AOD-Net) based on the channel pruning and INT8 quantization compression method for AOD-Net with improving computation speed and dehazing efficiency with no serious loss of accuracy. In addition, we find that E²AOD-Net with simple pruning attenuates the overfitting and strengthens the generalization ability, leading to a slight increase in the dehazing quality. PSNR which can evaluate the quality of dehazing, increased by an average of 0.9%, SSIM increased by 0.6% and MSE loss decreased by 1.8%.
- We design an energy-efficient accelerator based on the proposed method in this paper. The method includes multi-line buffer design (98.91% reduction in storage resources), pipeline parallel optimization design and ConvReLU PE design. The deployment experiments on PYNQ-Z2 shows that the energy efficiency is 42.61 GOPS/W, which surpasses other low-resource accelerators in recent years. The processing time of the accelerator is as short as 26 ms, with a power consumption as low as 2.491 watts, which far exceeds the comprehensive performance of CPU and GPU.
- With PYNQ framework for hardware-software co-design, we utilize Python to driver programs to achieve real-time inference on the accelerator and wrote reusable and stable Python code for continuous inference and IP reuse. Additionally, we conducted several experiments, including verifying the hardware inference effects and video testing, and exploring the potential performance loss compared to software. In these experiments, we find

that the discrepancies in MSE for hardware and software calculations are on the order of 10^{-8} , indicating our hardware design is correct. The video quality indicators such as VMAF (Video Multimethod Assessment Fusion) of the video after hardware dehazing are significantly improved (VMAF index is increased by 89.14%).

The rest of this paper is organized as follows. Section II provides the related work and relevant background knowledge of this paper. Section III provides the E²AOD-Net structure and accelerator design. Section IV provides experimental data and analysis. Section V concludes with a summary.

2 Preliminaries

2.1 CNN-based image dehazing

With the rise of deep learning, CNNs are gradually applied in various fields of computer vision and achieve better results than traditional algorithms. Cai [8] et al. firstly introduced CNNs into the field of image dehazing, their proposed Dehaze-Net can be trained to extract physical parameters from foggy images. Li [1] et al. proposed a lightweight end-to-end CNN-based image dehazing network AOD-Net, which is a network model that combines the transmittance and atmospheric light in the physical dehazing model into the same parameter, and uses only 5 convolutional layers, which has the advantage of being lightweight. Based on AOD-Net, Liu [27] et al. replaced ℓ_2 loss with perceptually derived loss functions (SSIM, MS-SSIM, etc.) and proposed PAD-Net. Qian [28] et al. proposed FAOD-Net using depthwise separable convolutions and adding pyramid pooling module, which significantly improves the dehazing quality and speed. H.Ullah [9] proposed a lightweight CNN called LightDehaze-Net(LD-Net), which further overcomes the traditional dehazing limitations, but the number of convolutional layers is still large.

2.2 AOD-Net

AOD-Net is based on the improved Atmospheric Scattering Model(ASM). The ASM model describes the image haze degradation process in the context of the haze imaging mechanism as:

$$I(x) = J(x)t(x) + A(1 - t(x)) \quad (1)$$

where $I(x)$ is the observed hazy image, $J(x)$ is the corresponding haze-free image, A is the atmospheric illumination intensity, and $t(x)$ is the medium transmittance. Li [1] et al.

¹ PYNQ-Z2 is a FPGA development board based on Xilinx ZYNQ SoC. It is designed to support PYNQ framework, and its core chip is xc7z020clg400-1.

unify the two parameters $t(x, y)$ and A into a single formula $K(x)$, constructing $K(x)$ as

$$K(x) = \frac{\frac{1}{t(x)}(I(x) - A) + (A - b)}{I(x) - 1} \quad (2)$$

Then the expression for $J(x)$ can be rewritten as

$$J(x) = K(x)I(x) - K(x) + b \quad (3)$$

This expression minimizes the reconstruction error, avoids the cumulative effect of errors that may occur when reconstructing the image in the previous methods, and optimizes the dehazing effect. The AOD-Net network model is used to estimate $K(x)$ to derive the dehazed image $J(x)$ from the hazy image $I(x)$. As shown in Fig. 1, we present the dehazing process and model structure of AOD-Net. In this paper we refer to Formula 3 in dehazing engineering as Post-Processing. The model has 5 convolutional layers and 3 connected layers. The concat1 layer concatenates features from the layers conv1 and conv2; similarly, concat2 concatenates those from conv2 and conv3; concat3 concatenates those from conv1, conv2, conv3, and conv4 [1]. Each convolutional layer has only 3 filters and the activation function uses the ReLU function.

2.3 Quantization

Quantization is a model compression algorithm that reduces the bit-width of a neural network parameters, which is essentially a mapping of floating-point numbers with a large range of values to fixed-point numbers with

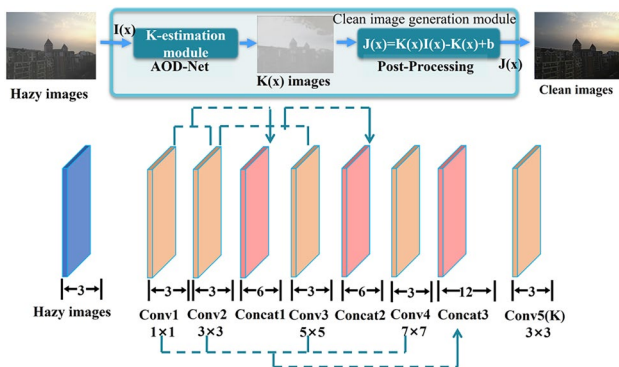
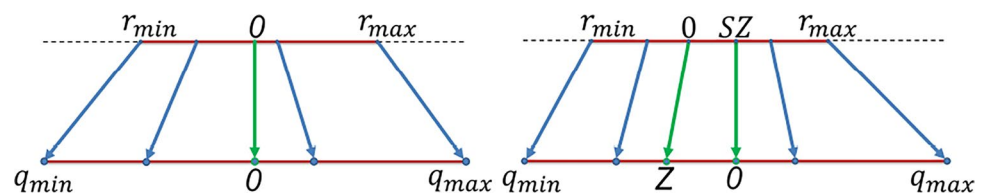


Fig. 1 Dehazing process and model structure of original design

Fig. 2 Symmetric and asymmetric quantitative mapping relation



a small range of values through a functional relationship. Quantization is classified into linear quantization and non-linear quantization according to the mapping function, and is classified into post training quantization(PTQ) and quantization-aware training(QAT) according to whether or not training is involved. Since nonlinear quantization introduces additional calculations in the conversion process, the industry generally adopts linear quantization methods with the mapping relation:

$$q = \text{round}\left(\frac{r}{S} + Z\right) \quad (4)$$

where r is the real number before quantization, q is the number of fixed-points after quantization, S is the scaling factor, Z is the zero point, and $\text{round}()$ is the rounding function. For $r \in (r_{\min}, r_{\max})$, $q \in (q_{\min}, q_{\max})$, there is $S = \frac{r_{\max} - r_{\min}}{q_{\max} - q_{\min}}$. The inverse process of quantization becomes dequantization, which is given by

$$r = S(q - Z) \quad (5)$$

The symmetric and asymmetric linear quantization mapping process is shown in Fig. 2, where the left side is symmetric quantization and the right side is asymmetric quantization. In this paper, asymmetric quantization is used for the input–output part of the model, while symmetric quantization is used for the parameter storage part, because zeroing zeros under symmetric quantization will reduce the computational cost of the inference process [3]. Since the $\text{round}()$ function needs to be used for rounding during quantization process, a certain quantization error will be introduced. QAT introduces fake-quantization nodes in the neural network during training, so the quantization error is considered during the training process. While according to [2], the PTQ method leads to significant accuracy degradation in small model. In this paper, we adopt the QAT method to compress the parameter width of our proposed algorithm to 8bit, see Sect. 3.1.2 for details.

2.4 Real-time image dehazing

The current demands for real-time dehazing images encompass low latency (more than 24 frame per seconds), high dehazing effectiveness, minimal power consumption, and portability. The hardware implementation choices include CPUs, GPUs, ASICs, and FPGAs. CPUs are known

to be unsuitable for edge computing scenario due to poor portability and high power consumption. Similarly, GPUs are also not ideal for edge computing due to their device size and power demands, despite their versatility. ASICs offer high performance with adequate power efficiency at the expense of high development cost and inflexibility of functionality. FPGAs, serving as a trade-off between ASICs and GPUs, are becoming a better choice for deploying dehazing algorithms with their balance of high efficiency and design flexibility, as well as agile development capability.

This paper focuses on FPGA deployment for dehazing algorithms, referencing past research where Zhang [10] et al. achieved 60fps video dehazing at 1920*1080 resolution on an FPGA, and G.Du [15] et al. implemented real-time dehazing at 38fps with 1280*960 resolution using image fusion techniques. Given the complexity of image dehazing systems, which handle acquisition, processing, and display, only FPGA may not suffice for complex control logic. Heterogeneous computing, combining different processors, is becoming more prevalent, with Xilinx ZYNQ SoC integrating CPU and FPGA for control and acceleration, respectively. So we adopt the PYNQ-Z2 development board as our platform. Previous studies, such as Wang [11] et al.'s deployment of AOD-Net using Vivado HLS, have demonstrated efficient dehazing at 2.4ms per image. However, their power consumption and resource consumption are high. While the tools FINN [26] and HLS4ML [25] developed based on HLS facilitate the deployment of neural networks on FPGAs, they do not optimize hardware resources as effectively as Verilog to ensure resource efficiency and performance. So in this paper, we adopt Verilog to design the accelerator IP.

3 Energy-efficient AOD-Net

The design uses low-cost PYNQ-Z2 as the deployment platform, and realizes a real-time video dehazing system through the collaborative optimization of algorithm and hardware. Firstly, AOD-Net is compressed by pruning and quantization to reduce resource overhead, and a new network model E²AOD-Net is proposed. Then, the flexibility of PYNQ framework and the parallelism of FPGA are used to design E²AOD-Net specific hardware accelerator, and a development process for deep neural network deployment application is explored in the process.

3.1 E²AOD-Net algorithm description

3.1.1 E²AOD-Net model structure

Due to the large number of parameters and complex calculations, neural network model AOD-Net require high hardware resources, which seriously limits their deployment and application on resource-constrained hardware platforms. Therefore, we use hardware-friendly pruning and INT8 quantization method for AOD-Net at the algorithmic level to reduce the number of intermediate feature maps that need to be stored in the inference process. In the original AOD-Net network, it takes a long time for the output feature maps of conv1 and conv2 to participate in the computation of conv5 through concat3, which leads to excessive memory resource overhead. We modified the concat3 layer to remove the connections from conv1 and conv2. Finally, we propose an E²AOD-Net(Energy-Efficient AOD-Net) model structure, as shown in Fig. 3, and the comparison table of parameters and storage quantities between AOD-Net and E²AOD-Net is shown in Table 1.

Fig. 3 E²AOD-Net model structure

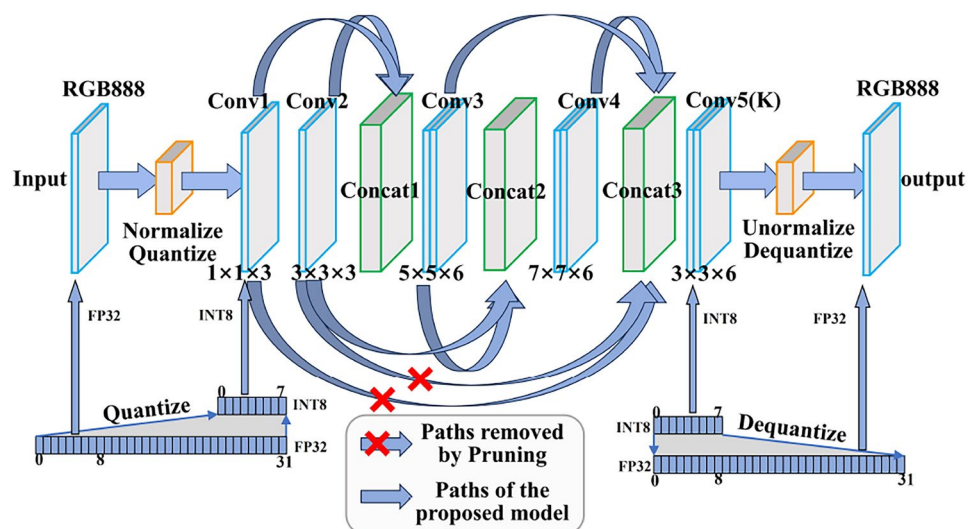


Table 1 Storage and parameter comparison between AOD-Net and E²AOD-Net

Model	Param	Memory cost (kB)
AOD-Net	1761 (float32)	6.88
E ² AOD-Net (FP32)	1599 (float32)	6.25
E ² AOD-Net	1584 (int8) + 33 (int16) ^a	1.61

^a The **data** in boldface indicates the parameters that are equivalent to the scale factors generated after quantization by the integer approximate floating-point strategy

The model parameters and storage capacity of E²AOD-Net(FP32) are reduced by 9.2% by model pruning. This optimization effectively reduces the computational complexity while maintaining the performance of the model. Further applying the quantization technique proposed E²AOD-Net, the reduction of storage space is more significant, reaching 76.6%. Through this series of optimizations, the deployment efficiency is significantly improved, the storage space requirements are reduced, and the resource consumption of model deployment is also reduced. Subsequent ablation experiments proved that the dehazing effect of E²AOD-Net did not decrease significantly, as detailed in 4.1 section.

3.1.2 Quantization method

The quantization method used in this paper is based on the linear INT8 quantization method proposed by Jacob [2] et al. for integer inference, and a integer approximate floating-point is performed to eliminate floating-point operations and to achieve integer-arithmetic-only inference. In addition, based on this quantization method, this paper quantize the post-processing process into INT8 data types for computation to reduce the hardware resource overhead of the Post-Processing. *Quantized Convolutional Layer Integer-arithmetic-only Inference Principles:* To reduce the additional operations introduced by the operation, the convolution weights and biases are symmetrically quantized, i.e., $r_w = S_w q_w$, $r_b = S_b q_b$; and asymmetric quantization is applied to the inputs and outputs, i.e., $r_x = S_x(q_x - Z_x)$, $r_y = S_y(q_y - Z_y)$. The convolution operation process can be expressed as

$$r_y^{i,k} = \sum_{j=0}^{N-1} r_x^{i,j} * r_w^{j,k} + r_b^{i,k} \quad (6)$$

Where x, y, w, b represent the input feature map, output feature map, convolution weight and bias respectively, $(i, j), (i, k), (j, k)$ represent the input feature map, output feature map, convolution weight coordinates respectively, and N is the number of input feature map channels. Bringing in the dequantize function yields

$$S_y(q_y^{i,k} - Z_y) = \sum_{j=0}^{N-1} S_x(q_x^{i,j} - Z_x) * S_w q_w^{j,k} + S_b q_b^{i,k} \quad (7)$$

After simplification the quantized output is obtained as

$$q_y^{i,k} = \frac{S_x S_w}{S_y} \left(\sum_{j=0}^{N-1} q_x^{i,j} * q_w^{j,k} - Z_x \sum_{j=0}^{N-1} q_w^{j,k} \right) + \frac{S_b}{S_y} q_b^{i,k} + Z_y \quad (8)$$

However, there are still $\frac{S_x S_w}{S_y}$ and $\frac{S_b}{S_y}$ which are in float32 data type, so we adopt the integer approximate floating-point strategy and set $\frac{S_x S_w}{S_y} \approx M_0 * 2^N$, the equivalent bias $B^{i,k} = \text{int}(\frac{S_x S_w}{S_y} (-Z_x \sum_{j=0}^{N-1} q_w^{j,k}) + \frac{S_b}{S_y} q_b^{i,k} + Z_y)$, and the quantized output equation is

$$q_y^{i,k} = \left(M_0 \sum_{j=0}^{N-1} q_x^{i,j} * q_w^{j,k} \right) \gg N + B^{i,k} \quad (9)$$

Convolutional computation of INT8 type data is achieved through quantization and integer approximate floating-point.

Quantized ReLU Integer-arithmetic-only Inference Principles: The ReLU function before quantization is

$$r_y = \max\{0, r_x\} = \begin{cases} 0, & r_x \leq 0 \\ r_x, & r_x > 0 \end{cases} \quad (10)$$

Substituting in the dequantization function yields

$$\begin{aligned} S_y(q_y - Z_y) &= \max\{0, S_x(q_x - Z_x)\} \\ &= \begin{cases} 0, & S_x(q_x - Z_x) \leq 0 \\ S_x(q_x - Z_x), & S_x(q_x - Z_x) > 0 \end{cases} \end{aligned} \quad (11)$$

The simplification yields a quantized ReLU output of

$$\begin{aligned} q_y &= \max \left\{ Z_y, \frac{S_x}{S_y}(q_x - Z_x) + Z_y \right\} \\ &= \begin{cases} Z_y, & q_x \leq Z_x \\ \frac{S_x}{S_y}(q_x - Z_x) + Z_y, & q_x > Z_x \end{cases} \end{aligned} \quad (12)$$

Where the inputs and outputs of a general ReLU have the same quantization parameters, i.e., $S_x = S_y$, $Z_x = Z_y$, further simplifying the computation of the quantized ReLU as

$$q_y = \begin{cases} Z_x, & q_x \leq Z_x \\ q_x, & q_x > Z_x \end{cases} \quad (13)$$

Quantized concat integer-arithmetic-only inference principles: Concat layer concatenate multiple feature maps, so the input feature maps need to have the same scale factor and zero point. For the problem that the quantization parameters of the output feature maps of different layers are different, the quantization factor unification node is introduced. The calculation process of the quantization factor unification node is dequantize first and then quantize,

i.e., dequantization to real numbers $r = S_x(q_x - Z_x)$, and then quantize $q_y = \frac{r}{S_y} + Z_y$ with the unified quantization, the above two-stage computation process introduces floating-point operations, which is not suitable for hardware implementation. In this paper, we combine the two equations to obtain

$$r = S_y(q_y - Z_y) = S_x(q_x - Z_x) \quad (14)$$

Simplification yields the quantized node output as

$$q_y = \frac{S_x}{S_y}(q_x - Z_x) + Z_y \quad (15)$$

where $\frac{S_x}{S_y}$ is a floating-point number, using the floating-point fixed-pointing strategy, set $\frac{S_x}{S_y} \approx M_1 * 2^N$. Eliminating floating-point operations by floating-point fixed-pointing approximation, the output of the node after quantization is

$$q_y = (M_1 * (q_x + (-Z_x))) \gg N + Z_y \quad (16)$$

Quantized Post-Processing Integer-arithmetic-only Inference Principles: According to the formula (3), the post-processing process can be decomposed into a combination of matrix point-wise multiplication, subtraction and addition. For the two inputs $r_{x1} = S_{x1}(q_{x1} - Z_{x1})$ and $r_{x2} = S_{x2}(q_{x2} - Z_{x2})$ of the multiplication and the outputs $r_{mul} = S_{mul}(q_{mul} - Z_{mul})$, the computation of the point-wise multiplication formula is $r_{mul} = r_{x1} * r_{x2}$, which, when brought into the quantization formula and simplified, yields

$$q_{mul} = \frac{S_{x2} * S_{x1}}{S_{mul}}(q_{x2} - Z_{x2}) * (q_{x1} - Z_{x1}) + Z_{mul} \quad (17)$$

Let $\frac{S_{x2} * S_{x1}}{S_{mul}} \approx M * 2^N$, then there are

$$q_{mul} = M(q_{x2} + (-Z_{x2})) * (q_{x1} + (-Z_{x1})) \gg N + Z_{mul} \quad (18)$$

Similarly, matrix subtraction in post-processing $r_{sub} = r_{x1} - r_{x2}$ is expressed as

$$q_{sub} = \frac{S_{x1}}{S_{sub}}q_{x1} + \frac{-S_{x2}}{S_{sub}}q_{x2} + \left(\left(-\frac{S_{x1}}{S_{sub}}Z_{x1} \right) + \frac{S_{x2}}{S_{sub}}Z_{x2} + Z_{sub} \right) \quad (19)$$

Let $\frac{S_{x1}}{S_{sub}} \approx M_{sub1} * 2^N$, $\frac{-S_{x2}}{S_{sub}} \approx M_{sub2} * 2^N$, $B_{sub} \approx (-\frac{S_{x1}}{S_{sub}}Z_{x1}) + \frac{S_{x2}}{S_{sub}}Z_{x2} + Z_{sub}$, Matrix subtraction after fixed-pointing by floating-point numbers is

$$q_{sub} = (M_{sub1}q_{x1} + M_{sub2}q_{x2}) \gg N + B_{sub} \quad (20)$$

The constant b in ASM model post-processing is usually set to 1. The matrix addition $r_{add} = r_x + 1$ can be expressed as

$$q_{add} = \frac{S_x}{S_{add}}q_x + \left(\frac{-S_xZ_x + 1}{S_{add}} + Z_{add} \right) \quad (21)$$

Let $\frac{S_x}{S_{add}} \approx M * 2^N$, $B_{add} \approx \frac{-S_xZ_x + 1}{S_{add}} + Z_{add}$, and the matrix addition after fixed-pointing of the floating-point numbers is

$$q_{add} = Mq_x \gg N + B_{add} \quad (22)$$

3.2 HW/SW co-design for E²AOD-Net accelerator

3.2.1 HW/SW co-design methodology

The proposed real-time dehazing system based on ZYNQ SoC with software and hardware collaboration is constructed, which takes full advantage of CPU flexible control and FPGA parallel acceleration. This design mainly consists of ARM Core, DMA, E²AOD-Net IP, HDMI TX IP, as well as off-chip DDR3 SDRAM, USB Camera, and HDMI Monitor. It adopts heterogeneous computing. ARM Core acts as the main processor and software end, which is also called PS (Processing System). In PS end, we write Python code to drive the AXI bus to control each component for image acquisition, preprocessing, DMA (simple way) sending and receiving data, and any other tasks appropriate for a software programming language. At the hardware end, namely PL (Programmable Logic), the AXI protocol is used to connect the related IPs to build a basic hardware system, and the E²AOD-Net IP serves as the processors for the computationally intensive image dehazing. The software and hardware co-operating system block diagram is shown in Fig. 4.

The processing flow of the real-time video dehazing system in this paper is as follows:

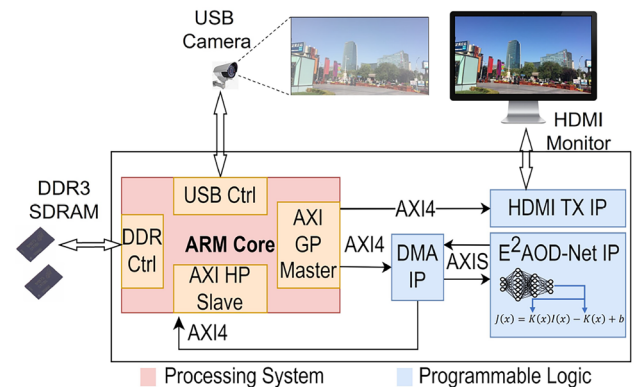


Fig. 4 Overall architecture of hardware accelerator system

- (1) ARM Core control USB Camera to read a frame of 640*480 resolution image and perform preprocessing.
- (2) Start the DMA to feed the preprocessed image into the E²AOD-Net Accelerator IP for inference.
- (3) ARM Core writes dehazed image to HDMI TX IP and the image is refreshed on the HDMI monitor.
- (4) Repeat (1)–(4) until the end of the image processing loop.

3.2.2 E²AOD-Net IP Design

E²AOD-Net IP uses a streaming architecture and consists of the ConvReLU PE, Multi-line Buffer, Quantized Concat, and Post-Processing module, its block diagram is shown in Fig. 5.

(1) ConvReLU PE module

ConvReLU PE uses strategies such as loop unrolling, weight reuse, pipeline and parallelism to complete the convolution inference operation with 8bit integer. A ConvReLU PE consists of a Processing Element (PE), a weight unit named Weights, a bias unit named Bias, and an activation function unit named ReLU and a control unit, the PE completes the MAC operation, Weights and Bias feed the kernel weights and bias parameters to the PE, and the ReLU completes the activation operation after the single-layer convolution operation. After completing the activation operation, its block diagram is shown in Fig. 6.

To accelerate the quantization convolutional inference deduced by Eq. (9), we adopt a loop unrolling to design PE, which expands the computational loop within the convolutional kernel into Multiply Accumulate(MAC) parallel computation, and is implemented using a tree-like structure to

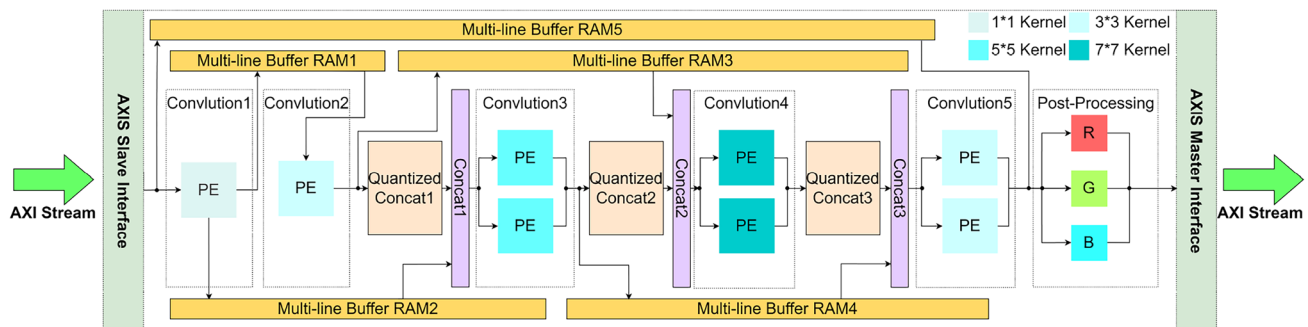


Fig. 5 Architecture of E²AOD-Net IP

Fig. 6 ConvReLU PE design structure

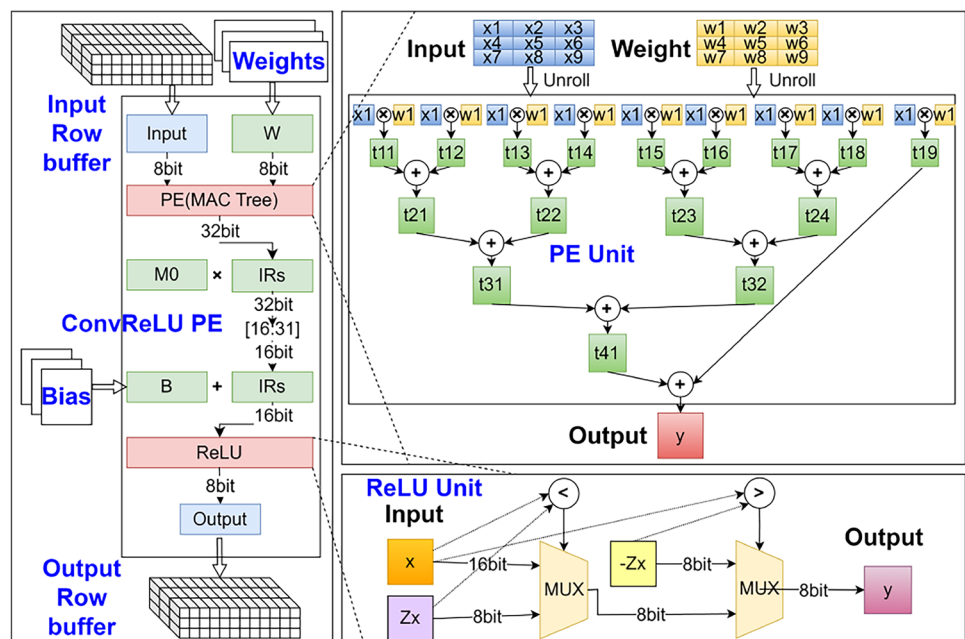


Fig. 7 The ConvReLU PE nested loop algorithm

Pseudocode for Traditional Convolution:

```

for(r=0; r < R; r++){//input row
  for(c=0; c < C; c++){//output col
    for(kr=0; kr < K; kr++){//kernel row
      for(kc=0; kc < K; kc++){//kernel col
        for(n=0; n < N; n++){//input channel
          for(m=0; m < M; m++){//output
            channel
            Out[m][r][c] += W[n][m][kr][kc] * In[m][S*r +
            kr][S*c + kc];
          }}}}}

```

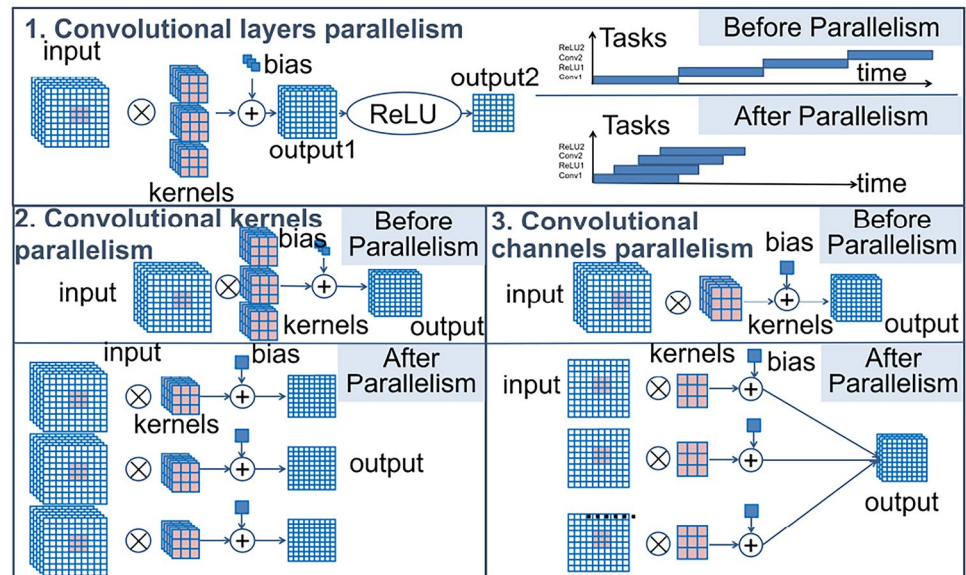
Input feature map: (N,H,L)
Weight: (M,N,K,K)
Output feature map: (M,R,C)

Pseudocode for input multiplexing:

```

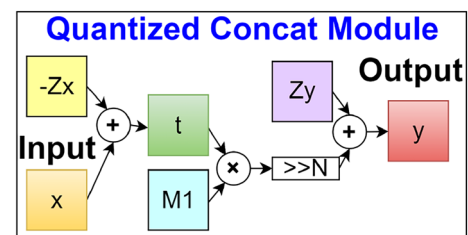
for(r=0; r < R; r+=Tr){//output row tiling
  for(c=0; c < C; c+=Tc){//output col tiling
    for(n=0; n < N; n+=Tn){//input channel tiling
      for(m=0; m < M; m+=Tm){//output channel tiling
        for(tr=0; tr < max(r+Tr,R); tr++){
          for(tc=0; tc < max(c+Tc,C); tc++){
            for(kr=0; kr < K; kr++){
              for(kc=0; kc < K; kc++){
                for(tn=0; tn < max(n+Tn,N); tn++){
                  for(tm=0; tm < max(m+Tm,M); tm++){
                    Out[tm][tr][tc] += W[tn][tm][kr][kc] *
                    In[tm][S*tr + kr][S*tc + kc];
                  }}}}}
                // Convolution accelerator internal calculation
              }}}}}
            }}}
          }}}
        }}}
      }}}
    }}}
  }}}
}

```

Fig. 8 Pipeline parallel optimization

further reduce the combinational logic path latency. Taking 3×3 convolution as an example, the schematic diagram of PE calculation is shown in the upper right corner of Fig. 6. For the quantized ReLU inference derived from equation (14), the subtraction result flag bit is used as the multiplexer selector signal and truncation is used to prevent data anomalies, which is structured as shown in ReLU Unit in Fig. 6. In this work, the weight data reuse strategy is used to reduce memory access delay, that is, until the sliding window convolution traverses the data of one input channel of the feature map before updating the data in the weight register for the convolution of the data of the next input channel, and its nested loop process is shown in Fig. 7.

To further improve the inference speed, this design adopts the strategy of inter-layer parallelism and inter-convolutional channel parallelism, i.e., the next layer of convolution does not need to wait for the end of the previous layer of convolutional computation before starting the computation, and there are multiple PEs inside the convolutional computation

**Fig. 9** Quantized concat design structure

to calculate the data of different channels at the same time, and the schematic diagram of the principle of parallelism is shown in Fig. 8. In the actual accelerator design, based on the above method, we implement the calculation method of RGB 3-channel parallel calculation (conv3 and conv4 are 6-channel parallel). Due to the limitation of resource, our convolution kernels use serial calculation to sacrifice the improvement of speed for the reduction of area.

(2) Quantized concat module

For the unified calculation of the quantization factor in the quantized Concat derived from the Formula 16, a pipelined approach is used to reduce the latency of combinational logic chain, which is structured as shown in the Quantized Concat Module in Fig. 9.

(3) Multi-line buffer for convolution layer module

The Concat operation in E²AOD-Net requires the output feature maps of different convolutional layers, so the intermediate feature maps need to be stored temporarily for subsequent computation. For large-size input feature maps, the length, width, and number of input channels are usually large, and caching the whole image on-chip consumes a large amount of storage resources, and due to the limited BRAM resources on the ZYNQ7020 SoC, it cannot store the feature maps for every convolutional layer in the model computation, and the speed of accessing the off-chip large-capacity DDR memory is limited by the access bandwidth while introducing additional power consumption, so the storage structure needs to be optimized.

According to sliding-window convolutional computation direction is left-to-right by line, we adopt the design approach of line buffer and design a dedicated line buffer structure for convolutional layers, through a special mapping between the hardware BRAM and the convolutional sliding-window, the state machine state and the output vector mask implement mapping and padding operations of different structures respectively. Based on the size of each convolutional kernel of the neural network multi-convolutional

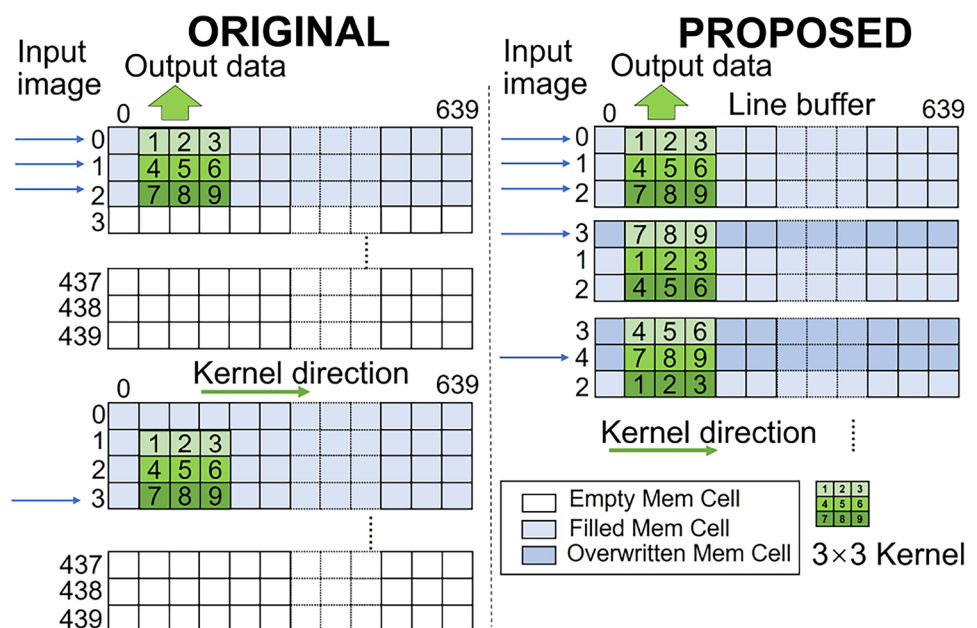
layer, an dedicated line buffer module is designed for each convolutional layer, which designs multi-line caches according to the size of convolutional kernel to meet the input data needed for the sliding window convolution. When a certain part of the input image data has been calculated, the subsequent input data can be overwritten by the RAM controlled by the line buffer module, thus achieving the purpose of saving cache resources. The schematic of the line buffer module for a convolution kernel size of 3*3 is shown in Fig. 10. The storage resource comparison Table 2 presents the comparison between our line buffer module and the original storage structure for the convolutional layer based on the input feature map size of 640*480. It is evident that our method achieves significant optimization in terms of required storage resources for all convolutional layers, except for the first layer. The overall reduction in cache resource usage amounts to 98.91%, which greatly facilitates hardware implementation.

(4) Post-processing module

Table 2 Comparison of storage resources between line buffering method and original

Layer	Kernel Size	Channel Input	Input Size(B)	Cache (kB)	
				Original	PROPOSED
Conv1	1*1	3	3	0	0
Conv2	3*3	3	27	900	5.625
Conv3	5*5	6	150	1800	20.625
Conv4	7*7	6	294	1800	30
Conv5	3*3	3	54	1800	46.875

Fig. 10 Multi-line buffer optimization for Convolutional layer



The post-processing calculation module is specifically designed based on the Formula 3. It aims to further process the output feature map of E²AOD-Net to obtain a high-quality image after dehazing. In this paper, based on Formula (16, 18, 20), we design a dedicated post-processing module to achieve the hardware acceleration of the formula calculation. To reduce the combinational logic chain demonstration and improve the system throughput, this paper inserts 5-stage pipeline registers in the Post-processing module.

4 Experiments

In this paper, the algorithm deployment environment is based on AMD Xilinx's FPGA development software vivado2022.1. The PC configuration is an Intel Core i913900HX CPU with a main frequency of 5.4 GHz and an NVIDIA RTX 4060 GPU. the actual verification platform of the algorithms is a PYNQ-Z2 development board, a USB camera, and an HDMI monitor. The algorithm training and testing environment for the related software algorithms is based on PyTorch2.1.1 and Python3.11.5, and the model quantization training part are Tensorflow2.10.0 and Python3.9.18. The size of images used for testing and training in the experiments is 640*480.

4.1 Experiments on model generalization ability of E²AOD-Net

To investigate whether the inference capability of the E²AOD-Net(FP32) model has decreased, we conducted a dehazing inference experiment. To make the inference ability of the dehazing model more visual and intuitive, we choose to use three objective average metrics obtained by comparing the dehazed image with the haze-free image: PSNR, SSIM, and MSE. PSNR is an important indicator of signal reconstruction quality in fields such as image compression, the larger the value, the smaller the image quality degradation and less distortion. SSIM is used to measure the similarity of the structural information between the two images, the larger the value, the higher similarity between the image and the original image. MSE is used as an indicator that measure the difference between the predicted value and the ground truth value, the smaller its value, the more accurate the predicted value is.

Subsequently, we conducted a comparative experiment to assess the model's inference capability between AOD-Net and E²AOD-Net. To ensure the reliability of our experimental results, we employed the control variable method, where only the network model structure varied during the training and inference processes of both models while keeping other parameters and settings consistent.

Subsequently, we evaluated model inference accuracy by assessing multiple test sets using objective evaluation metrics. These test sets comprised, D-HAZY [20] (deep haze), NH-HAZE [21] (natural haze), SOTS [22](thin haze), HSTS [22] (thin haze), and FRIDA [23] (fuzzy haze) datasets. The D-HAZY dataset consists of images portraying scenes with deep haze; whereas NH-HAZE includes various types of hazy scene images occurring in natural environments. SOTS includes images with slight synthetic processing-induced haze effects in various outdoor scenes, while HSTS focuses more on real-world hazy scenes collected from actual environments with slight haziness present. SOTS and HSTS test sets are the two most similar to the training set, so the relevant evaluation indicators have the highest reference value on these two test sets. The FRIDA test set primarily serves for evaluating generated hazy effects and consists of images featuring blurred fog created through a specific algorithm for assessing inference performance. We carefully selected distinct hazy and haze-free images for corresponding objective metric evaluations to ensure that these datasets were not encountered during the model training process. Finally, average differences in inference performance before and after pruning were recorded and compared. The difference results are shown in Table 3.

After experimental verification, it was found that the pruned model E²AOD-Net performed better than the original model AOD-Net on other test sets. Particularly, on the SOTS and HSTS thin haze test sets, the PSNR indicators improved by 0.29 and 0.33 respectively, while the SSIM index was slightly improved, and the MSE index was slightly decreased. These experiments demonstrate that the pruned model has better generalization ability after channel pruning, significantly preventing overfitting during deep neural network training, exceeding expectations. And Lorenz Kuhn et al. have come to the following conclusions through numerous experiments that the smaller the fraction of parameters a model actually "uses" - the simpler it is - the better the network's generalization performance [24]. The result is consistent with the conclusion drawn in this experiment.

Table 3 Comparison of model inference accuracy between AOD-Net and E²AOD-Net

Dataset	PSNR↑ (dB)	SSIM↑	MSE↓ (%)
	AOD/E ² AOD	AOD/E ² AOD	AOD/E ² AOD
D-HAZY	11.71/ 11.82^a	0.732/ 0.734	1.927/ 1.899
NH-HAZE	12.01/ 12.05	0.461/ 0.470	1.429/ 1.421
SOTS	23.33/ 23.62	0.883/ 0.886	0.133/ 0.125
HSTS	22.66/ 22.99	0.888/ 0.889	0.169/ 0.168
FRIDA	15.26/ 15.31	0.832/ 0.835	0.663/ 0.658

^a The **data** in boldface indicates their superiority in the data comparison

4.2 Software dehazing algorithm experiments

To comprehensively evaluate the effect and practicality of the algorithm, this study designed a series of experiments to compare and analyze the algorithm proposed in this paper with traditional dehazing algorithms. In the comparison experiments of neural network based dehazing algorithms, to ensure that the performance differences between different neural network are not affected by the training hyper parameters, we use a unified training hyper parameter configuration for all the neural networks involved in the comparison. The specific parameters were set as follows: the number of training epochs was fixed at 10, the optimizer was chosen as Adam, the learning rate was set at 0.0001, the weight decay was also set at 0.0001, the loss function was adopted as the MSE loss function, and the training dataset was RESIDE [22], which includes a total of 335,734 hazy images (ratio of indoor and outdoor images is 1:9) from the NYU2 series and OTS series, as well as 10,319 haze-free images corresponding to each.

To ensure the fairness and reliability of the experimental results, we randomly selected 50 indoor and 450 outdoor hazy images and their corresponding haze-free images, a total of 500 pairs of images, based on the screened RESIDE dataset, for evaluating and testing the performance of different software algorithms. The experimental results of the subjective results are shown in Fig. 11, while the experimental results for objectively evaluation indicators are summarized in Table 4. Through these rigorous experimental designs and detailed result analyses, this paper aims to provide strong evidence for the effectiveness and feasibility of the proposed algorithms.

Table 4 Objective evaluation criteria and average processing time for various software algorithms

Ref.	Algorithm	Time (s)	PSNR (dB)	SSIM	MSE(10^{-2})
ICCV'09 [12]	DCP	0.040^a	16.47	0.810	0.605
TOMM'21 [15]	Fusion-based	1.893	14.94	0.759	0.804
WACV'19 [13]	GCA-Net	0.485	23.11	0.892	0.165
MVA'21 [14]	DC-Net	0.431	22.70	0.872	0.230
TIP'21 [9]	LD-Net	5.437	22.93	0.879	0.160
ICCV'17 [1]	AOD-Net	0.356	22.71	0.871	0.174
	Proposed(FP32)	0.401	23.00	0.874	0.166
	Proposed(INT8)	1.895	22.79	0.872	0.178

^a The **data** in boldface indicates the top three most outstanding ones under this column

In the experiment, we trained TFlite (Tensorflow lite) network models with FP32 precision and INT8 precision based on the proposed network E²AOD-Net. To facilitate the hardware implementation of the algorithm, we extracted the weight parameters from these two TFlite models and respectively wrote neural network convolution layer functions, Concat layer functions, and image post-processing functions with FP32 and INT8 precision using the numpy library in Python language to simulate hardware computation methods. P_FP32 and P_INT8 are models written based on the aforementioned functions and parameters.

The experimental results show that, in terms of subjective evaluation, the non-neural network dehazing algorithm has



Fig. 11 Dehazing effect of different software algorithms

a certain dehazing effect on the image. Since the algorithm proposed in fusion-based dehazing [15] includes a part of the dark channel dehazing, its dehazing effect is similar to that of DCP. The dehazed image is overall darker in tone compared to the original image, and the handling of environmental details is not very accurate. On the other hand, neural network-based algorithms demonstrate better dehazing effects and superior handling of image details compared to the aforementioned algorithms. Among them, the dehazed images produced by DC-Net are subjectively perceived to have poorer effects, and GCA-Net performs poorly in handling details of the sky background. The dehazed images produced by the remaining algorithms are difficult to distinguish from the original images.

In terms of objective evaluation, we compared the images processed by each algorithm with the haze-free image from four dimensions: single-frame processing time, PSNR, SSIM and MSE. The average single-frame processing time is calculated in Python, the shorter the processing time, the higher the more lightweight the algorithm. After the detailed introduction of 4.1, we have fully discussed the indicators of PSNR, SSIM and MSE. Therefore, we will not elaborate on them in the following content.

According to Table 4, non-neural network algorithms DCP and the algorithm proposed in fusion-based dehazing [15] exhibit poor performance in terms of objective metrics. Although the DCP algorithm has the shortest processing time in the table, accounting for only 11.2% of the AOD-Net algorithm, it is disregarded due to its overly simplistic structure and inadequate dehazing effect. On the other hand, among the neural network-based dehazing algorithms (GCA-Net, DC-Net, LD-Net, and AOD-Net), although AOD-Net algorithm used in this paper ranks third among the 4 algorithms in terms of PSNR, SSIM, and MSE, other algorithms require longer processing times, being 1.36, 1.21, and 15.27 times that of AOD-Net, respectively. Therefore, overall, AOD-Net shows superior dehazing performance while being lightweight. The proposed algorithm P_INT8 based on E² AOD-Net in this paper shows a slight decrease in objective metrics PSNR and SSIM compared to the FP32 algorithm, while its processing time significantly increases. To explore this, we measured the processing time of the quantization related functions inside the P_INT8 model. We found that the conv4 layer (kernel size of 7*7) takes an average of 1.1 s, accounting for 58% of the total processing time. Further exploring, all quantization related functions have more or less increased processing time compared to P_FP32 functions. Therefore, the slower performance of the INT8 model can be attributed to the increased computational overhead from quantization operations. Although pruning and quantization may impact the model's accuracy and introduce unexpected distortion during the image processing, considering its lightweight nature, it is highly energy-efficient. Moreover,

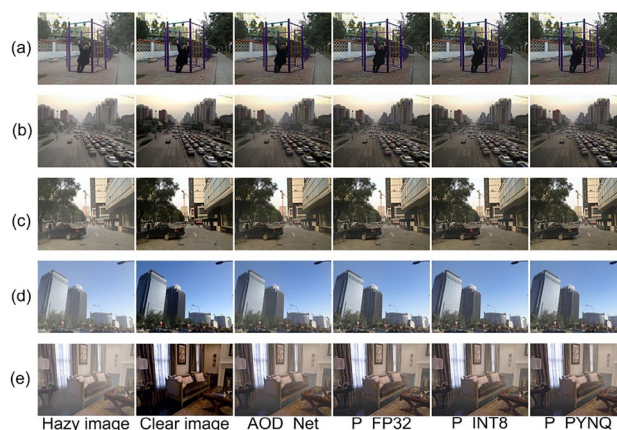


Fig. 12 Dehazing effect of accelerator and software algorithm dehazing

Table 5 Image evaluation metrics and time consumption for software and hardware deployment

Algorithm	Time(s)	PSNR(dB)	SSIM	MSE (10^{-2})
AOD-Net	0.356	22.71	0.870	0.175
Proposed (FP32)	0.401	23.00	0.874	0.166
Proposed (INT8)	1.895	22.79	0.872	0.178
Proposed (PYNQ)	0.026	22.79	0.872	0.178

it is challenging to subjectively perceive differences between various dehazing algorithms in image dehazing. Hence, we selected the P_INT8 algorithm for hardware implementation and validated its effectiveness in hardware-based dehazing through subsequent experiments.

4.3 Hardware accelerator experiments

4.3.1 Hardware output vs. software output

According to Fig. 4 of the real-time video dehazing system deployed in Zynq SoC, we use the PYNQ framework which can use Python as programming language to drive the FPGA IP. We store 500 pairs of test images of size 640*480 in the SD card, and use the same as the software algorithm test of the Python code for the test, resulting in Zynq PL of the implementation of the dehazing algorithm processed by the image, the shape of software algorithm image metrics evaluation, subjective image experimental results as shown in Fig. 12, objective evaluation metrics experimental results as shown in Table 5. P_PYNQ is the hardware accelerator that implements E²AOD-Net on the PYNQ development board.

The experimental results show that, in terms of subjective evaluation, there is no significant difference in the image dehazing effect between the hardware implemented PYNQ

algorithm and the software algorithm, and the processed image does not have obvious distortion. In terms of objective evaluation, the hardware implemented algorithm demonstrates a significant advantage in single-frame image processing time, completing the process in just 26 ms, which means its frame rate is 38.3fps. Its processing speed is 72.885 times faster than the P_INT8 algorithm running on the CPU, meeting the real-time image dehazing requirements. Additionally, through the comparison and analysis of image quality evaluation indicators such as PSNR, SSIM, and MSE, we found that the hardware implementation algorithm is almost consistent with the results calculated by P_INT8. We also calculate the MSE error of PYNQ and P_INT8, and the value is 1.88×10^{-8} , which shows the correctness of our hardware design. In terms of data comparison, compared with the FP32 precision model, the objective indicators did not decrease sharply and there was no obvious distortion.

4.3.2 Comparison of proposed implementation on different platform

In this experiment, we compared and evaluated the performance of three different deployment methods of FPGA, CPU and GPU on neural networks. Through this comparison, we aim to explore the impact of different deployment on neural network performance, to express the advantage of FPGAs that can process data on a large scale

Table 6 Algorithm power consumption and time comparison across different platforms

Hardware	Platform	Power (W)	Time (s)
GPU	NVIDIA GeForce RTX 4060	125	0.011
CPU	Intel Core i9-13900HX	55	1.895
Proposed	Zynq 7020	2.491 ⁽¹⁾	0.026

⁽¹⁾ Power of the entire video processing system

and consume less power. Table 6 shows the comparison table of total power consumption and inference time for different platforms.

We conducted an in-depth analysis of the performance of the same neural network-based dehazing algorithm deployed on PYNQ and compare it with GPU. Experimental results show that although the PYNQ deployment takes 2.6 times longer than GPU to process a single frame image, it is still at millisecond level, and its power consumption is only 1.9% of GPU. In addition, compared with CPU, PYNQ deployment show significant advantage in power consumption and processing time, achieving all-round performance transcendence. These results prove the excellent performance of the hardware platform proposed in this paper in terms of cost performance, which can not only meet the needs of real-time image dehazing, but also maintain extremely low power consumption levels. Therefore, the hardware platform used in this study provides an efficient and energy-saving solution for FPGA implementation of Deep Learning algorithms.

4.3.3 Comparison of proposed accelerator with others

After a comprehensive comparison of the performance of algorithms deployed on three different hardware platforms: CPUs, GPUs and FPGAs, the significant advantages of the FPGA platform are highlighted. Based on this, this research further investigates the deployment performance of the designed neural network IP on multi-FPGA platforms, aiming to highlight the outstanding performance of the algorithms proposed in this paper in terms of hardware implementation on FPGA. To show in detail the comparative performance of neural network deployment on multi-FPGA platforms, this paper provides exhaustive performance evaluation data as shown in Table 7. Based on the description of Flops in [29], the equation for calculating the GOPS of the key metrics in the table is provided here

Table 7 Implementation and performance comparison of FPGA-based accelerators

	Hardware/ Software	Platform	Data Format	Frequency (MHz)	Power (W)	Energy efficiency (GOPS/W)	Num. of resource			
							LUT	FF	DSP	BRAM
FPGA'16 [4]	Hw&Sw	Xc7z045	INT16	150	9.63	19.55	182,616	127,653	780	486
DAC'17 [5]	Hw&Sw	Xc7z045	INT16	100	9.4	24.42	155,886	120,957	824	909
XDU'22 [16]	Hw&Sw	Xc7a200t	INT8	200	4.695	30.02	117,102	113,181	360	5440
CAC'22 [18]	Hw&Sw	Xc7z020	INT16	150	2.609	10.37	35,373	32,555	149	175
IMCEC'22 [19]	Hw&Sw	Xc7z020	INT16	150	2.812	15.05	53,000	30,752	150	114
TCAD'18 [6]	Hw&Sw	Xc7z020	INT16	214	3.5	24.09	29,867	35,489	190	85.5
CW'21 [17]	Hw&Sw	Xc7z020	FP32	200	1.685	34.18	28,818	34,761	N/A	30
TOMM'21 [15]	Hw	Xc6vlx240t	INT8/16	89	3.469	N/A	25,155	9127	N/A	N/A
Proposed	Hw&Sw	Xc7z020	INT8	113	0.893	42.61	23,242	21,504	20	37

$$GOPS = (C_i * (K_h * K_w) * C_o) * H * W * 2 * n \quad (23)$$

where C_i and C_o is the number of input and output channels, K_h and K_w are the height and width of the convolution kernel, H and W are the height and width of the output image, n is the times of model processing in a second.

In the table, we select two categories of FPGA deployments for comparison. One category represents large-scale embedded systems that utilize a significant amount of hardware resources, while the other category represents small-scale embedded systems with fewer hardware resources. By comparing the two FPGA implementations, we aim to explore the advantages of our proposed design. Additionally, it is worth noting that all references, except for [15], focus on the deployment of neural networks on FPGA. Although [15] does not specifically address neural network deployment, it presents an FPGA implementation of a dehazing algorithm within the same domain as our study, making its data valuable for reference. Through this comparative analysis, our objective is to demonstrate that our proposed algorithm not only exhibits high performance advantages in FPGA deployment but also demonstrates excellent scalability and flexibility. This research provides valuable insights and guidance for the design of future deep learning hardware accelerators.

In this study, we have designed a CNN FPGA accelerator specifically for 8-bit integer inference and comprehensively analyzes its energy efficiency based on 640*480 image test experiments. The experimental results demonstrate that our proposed neural network IP exhibits lower power consumption, at only 7.63% of the power consumption of the highest efficient and best-performing reference [16] in large-scale embedded platforms. Furthermore, our design achieves higher energy efficiency, at 1.42 times that of the aforementioned reference. Notably, the maximum clock frequency required for our accelerator is 113 M, and it consumes only 19.8% of the LUT, 18.9% of the FF trigger, 5.6% of the DSP, and a mere 0.7% of the BRAM compared to the aforementioned literature. These exceptional performance metrics highlight the superiority of our design.

In particular, on the same platform, namely the Xc7z020 chip, the accelerator proposed in this study demonstrates the highest energy efficiency among recent research findings. Compared to the reference [17], which achieves the highest energy efficiency on small embedded platforms, the accelerator in this paper is 1.25 times larger. Moreover, it exhibits more economical utilization of hardware resources: the consumption of LUTs is reduced by 19.4%, the occupancy of FFs is decreased by 38.4%, and there is a slight increase of 23.3% in the occupancy of BRAM resources. These results fully showcase the advantages of the designed CNN FPGA accelerator in terms of energy efficiency and cost-effectiveness. Compared to other FPGA accelerators

that also deploy neural networks, our energy efficiency is higher and the power consumption is lower, which is the advantage of our Verilog design accelerator.

4.4 Real-time video quality assessment experiments

We build a real-time video dehazing system on the PYNQ-Z2 development board, which utilize 36,232 LUTs, 43,794 FFs, 56 DSPs, and 49 BRAMs. Then we obtain two dehazed videos of an identical video form PYNQ board and python code that simulates hardware at the algorithmic level(P_INT8) respectively, The quality of these two videos is now evaluated using some Video Quality Assessment (VQA) metrics. Since the original video is a real scene shooting, it is not possible to get the ideal haze-free video. To facilitate the video evaluation, a dehazed video obtained through the trained optimal TFlite model is used to simulate the haze-free video as a reference for FR-VQA(Full-Reference Video Quality Assessment).

TI is a BVQA (blind video quality assessment) metric, which indicates the number of temporal variations in a video sequence, the smaller the metric, the smaller the inter-frame variation of the video.since it is a non-reference indicator, the results are greatly influenced by the original video scene motion and changes; VQM is an objective VQA metric that takes into account human's visual perception, the smaller the value, the higher the quality of the video; VMAF is a kind of modern reference metric that consist of 4 features (ADM, VIF, Motion, ANSNR) that are bigger better quality, in this experiment we use the reference values obtained after synthesizing the features. These metrics were computed using the MSU Video Quality Tool. The work done in the experiment is as follows:

1. Video Temporal Consistency of the two de-noised videos was evaluated using TI metric.
2. The comprehensive quality of two dehazed videos was evaluated using VQM and VMAF metric. The result is compared side-by-side to investigate the performance loss of the dehazing hardware implementation compared to software.

Table 8 Evaluation and comparison of dehazing video quality of the hardware accelerator and software

Videos	TI↓	VQM↓	VMAF↑
Haze-free Video	0.020	N/A	N/A
Haze Video	0.018	6.37	40.97
Proposed (INT8)	0.026	3.55	79.45
Proposed (PYNQ)	0.032	4.16	77.49

The evaluation results is shown in Table 8. For the TI metrics, their values have increased after the dehazing process, indicating a decrease in Video Temporal Consistency, which is normal because even the onnx model has some increase in TI metrics after dehazing. Comparison of the results with the onnx model TI by visualization tools shows that the main loss of Video Temporal Consistency is caused by the inevitable noise generated in the dehazing and the dehazing distortion of the original hazy video in the transitions. Loss of hardware implementation only increases by about 11% over the results simulated by software, and the difference is not significant. For the VQM and VMAF metrics, the hardware implementation decreases by about 17% and 2.4% respectively compared to the results simulated by software, which is within a acceptable range, and both of their metrics values are greatly improved compared to the original hazy video, realizing a relatively good video dehazing effect.

5 Conclusion

In this paper, we adopt the idea of algorithm and hardware co-design, and prune and quantify the AOD-Net dehazing neural network at the algorithm level, and propose a new model named E²AOD-Net. The new network structure has enhanced the generalization ability compared with the original network model; at the hardware level, we design a dedicated hardware accelerator using optimization methods such as loop unrolling, pipeline parallel, and line caching, and build a real-time video dehazing system based on the PYNQ framework on the ZYNQ heterogeneous SoC, and study the performance loss of hardware video dehazing and software comparison. Our image dehazing IP can process images with a resolution of 640*480 at a frame rate of 38.3 fps, achieving a energy efficiency of 42.61 GOPS/W. The entire system consumes only 2.491 W of power. The accelerator has the advantage of high energy efficiency while maintaining high processing efficiency, significantly reducing energy consumption and proving its great potential in real-time processing and low-cost application scenarios.

Author Contributions Z.Z. and G.D. wrote the main manuscript text and Z.Z. and Z.L. prepared all the figures. Q.K. and W.Z. help build the experimental environment. X.W. supervised the experimentation as well as the manuscript composing. All authors reviewed the manuscript.

Data availability statement No datasets were generated or analyzed during the current study.

Declarations

Conflict of interest The authors declare that there is no conflict of interest.

References

1. Li, B., Peng, X., Wang, Z. et al.: Aod-net: All-in-one dehazing network. In 2017 IEEE International Conference on Computer Vision (ICCV), pages 4780–4788 (2017)
2. Jacob, B., Klugys, S., Chen, B., et al.: Quantization and training of neural networks for efficient integer-arithmetic-only inference. In 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 2704–2713 (2018)
3. Gholami, A., Kim, S., Dong, Z. et al.: A survey of quantization methods for efficient neural network inference. [arXiv:2103.13630](https://arxiv.org/abs/2103.13630) (2021)
4. Qiu, J., Wang, J., Yao, S. et al.: Going deeper with embedded FPGA platform for convolutional neural network. In Proceedings of the 2016 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays, FPGA '16, pp. 26–35, New York, NY, USA (2016). Association for Computing Machinery
5. Xiao, Q., Liang, Y. Lu, L. et al.: Exploring heterogeneous algorithms for accelerating deep convolutional neural networks on FPGAs. In: 2017 54th ACM/EDAC/IEEE Design Automation Conference (DAC), pp. 1–6 (2017)
6. Guo, K., Sui, L., Qiu, J., et al.: Angel-eye: a complete design flow for mapping cnn onto embedded FPGA. *IEEE Trans. Comput. Aided Des. Integr. Circ. Syst.* **1**, 35–47 (2018)
7. Koranga, P., Kumawat, S.: A review on comparison of different techniques of image dehazing. In 2020 5th IEEE International Conference on Recent Advances and Innovations in Engineering (ICRAIE), pp. 1–6 (2020)
8. Cai, B., Xiangmin, X., Jia, K., et al.: Dehazenet: an end-to-end system for single image haze removal. *IEEE Trans. Image Process.* **25**(11), 5187–5198 (2016)
9. Ullah, H., Muhammad, K., Irfan, M., et al.: Light-dehazenet: a novel lightweight cnn architecture for single image dehazing. *IEEE Trans. Image Process.* **30**, 8968–8982 (2021)
10. Zhang, C.L., Xu, R., Wang, Y.J., et al.: FPGA-based video real time dehazing algorithm and hardware implementation[J]. *Semiconductor Optoelectron.* **42**(2), 264–268, 274 (2021)
11. Wang, G., Jian, C., Xiang, Q.: Hardware reconstruction acceleration method of convolutional neural network-based single image dehazing model. *J. Comput. Appl.* **42**(10), 3184–3190 (2022)
12. He, K., Sun, J., Tang, X.: "Single image haze removal using dark channel prior. In: 2009 IEEE Conference on Computer Vision and Pattern Recognition, Miami, FL, 1956–1963 (2009). <https://doi.org/10.1109/CVPR.2009.5206515>
13. Chen, Dongdong, He, Mingming, Fan, Qingnan, et al.: Lu Yuan, and Gang Hua. Gated context aggregation network for image dehazing and deraining. In 2019 IEEE Winter Conference on Applications of Computer Vision (WACV), pages 1375–1383 (2019)
14. Bhola, Akshay, Sharma, Teena, Verma, Nishchal K.: Dcnnet: Dark channel network for single-image dehazing. *Mach. Vision Appl.* **32**(3) (may 2021)
15. Du, Gaoming, Wu, Jiting, Cao, Hongfang, et al.: A real-time effective fusion-based image dehazing architecture on FPGA. *ACM Trans. Multimedia Comput. Commun. Appl.*, **17**(3) (jul 2021)
16. Kezheng, C.: Research on real image dehazing algorithm based on convolution neural network and hardware acceleration technology.

- Xidian University (2022). <https://doi.org/10.27389/d.cnki.gxadu.2022.001996>
17. Alaeddine, Hmidi, Jihene, Malek, Khemaja, Maha: An efficient deep network in network architecture for image classification on FPGA accelerator. In 2021 International Conference on Cyber-worlds (CW), pages 72–77, (2021)
 18. Zhan, Canyu, Zhao, Bingjie, Zhang, Huangliang, et al.: Design and implementation of FPGA-based hardware acceleration system for target detection. In 2022 China Automation Congress (CAC), pages 553–559 (2022)
 19. An, Sheng, Deng, Jijie: Low-resource cnn accelerator based on FPGA. In 2022 IEEE 5th Advanced Information Management, Communicates, Electronic and Automation Control Conference (IMCEC), volume 5, pages 1229–1233 (2022)
 20. De Vleeschouwer Cosmin Ancuti, Christophe, Ancuti, Codruta O.: D-hazy: A dataset to evaluate quantitatively dehazing algorithms. In IEEE International Conference on Image Processing (ICIP), ICIP'16, (2016)
 21. Ancuti, Codruta O., Ancuti, Cosmin, et al.: NH-HAZE: an image dehazing benchmark with non-homogeneous hazy and haze-free images. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops, IEEE CVPR 2020, (2020)
 22. Li, B., Ren, W., Dengpan, F., et al.: Benchmarking single-image dehazing and beyond. *IEEE Trans. Image Process.* **28**(1), 492–505 (2019)
 23. Tarel, Jean-Philippe, Hautière, Nicolas, Cord, Aurélien, Halmaouiet, Houssam, et al.: Improved visibility of road scene images under heterogeneous fog. 2010 IEEE Intelligent Vehicles Symposium, pages 478–485, (2010)
 24. Kuhn, Lorenz, Lyle, Clare, Gomez, Aidan N, et al.: Robustness to pruning predicts generalization in deep neural networks. *ArXiv*, abs/2103.06002 (2021)
 25. Fahim, Farah, Hawks, Benjamin, Herwig, Christian, et al.: hls4ml: An open-source codesign workflow to empower scientific low-power machine learning devices, (2021)
 26. Umuroglu, Y., Fraser, N.J., Gambardella, G. et al.: Finn: A framework for fast, scalable binarized neural network inference. In: Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays, FPGA '17. ACM (2017)
 27. Liu, Y., Zhao, G.: P Pad-net: A perception-aided single image dehazing network. *arXiv:1805.03146* (2018)
 28. Qian, W., Zhou, C., Zhang, D.: Faod-net: A fast aod-net for dehazing single image. *Math. Probl. Eng.* **2020**(1), 4945214 (2020)
 29. Teng, P., Du, G., Li Z. et al.: High-speed hardware accelerator based on brightness improved by light-dehazenet. *J. Real-Time Image Process.* **21**(3) (2024)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.